Apartment 50: Paper 006

AuthorityEffect: None. The scene is fictional examination staging, not a historical quotation, a reconstruction of Landau's examinations, or a source of mathematical authority. Levels 0-7 are this companion's curriculum.

Landau asks what was actually retained. Woland asks who holds the anchor. Behemoth removes the last line and rewrites the header. The record still opens; the examiner requires an explanation of what that proves.

Start with the study guide, then attempt the quiz without the instructor key. The key includes the grading standard and an exact-arithmetic replay. Read the manuscript alongside the exercises; its historical source pin governs implementation claims. A current checkout and a historical audit are distinct inputs.

The standard is a calculation, a concrete counterexample, or an explicitly missing obligation. A stored refusal does not settle its mathematical meaning; a digest does not authenticate its producer; an emitted decision record does not show that a host acted on it. These are the questions to answer before claiming reusable knowledge.

Study guide: Paper 006

AuthorityEffect: None. Use this guide before the quiz; use the separate instructor key afterward. The preface explains the fictional scene. Prerequisites are JSON objects and strings, exact fractions, a predecessor chain, elementary hashing, and the difference between a returned value and a side effect. No biological expertise is required.

Reading map

Level	Learn to do	Manuscript
0	Name event, storage, and authority claims separately	Sections 1-3
1	Trace normalization and the actual encoding profile	Sections 3-4
2	Compare a verified snapshot with an independent anchor	Sections 5-6
3	Analyze failed writes and bounded resources	Sections 4-6
4	Recompute the catalog census without promoting labels	Section 7.1
5	Interpret clonal returns and toxicity denominators	Sections 7-8
6	Calculate decay, expiry, and stimulation	Section 8
7	Specify reuse and an honest publication receipt	Sections 7, 9-10

1. Retained information and a genuine external event

The sink records information supplied by its caller. A caller may label a refusal `PROTECTED_PATH`, but the sink does not inspect ownership rules and recompute that decision. Store identity and decision correctness are separate checks. A native-result claim needs the actual verifier, its source or binary, input/output bytes, and an evaluator for their relationship.

Similarly, the literal marker `"None"` must be present under the declared contract. Its correctness is a field check. It is not a signature or a grant of authority. A

refusal can be useful even when the cause is absent infrastructure rather than an invalid mathematical object.

2. Normalize before reasoning about hashes

At the audited commit, fractions become rational strings; mapping keys become strings; tuples become lists. This is a storage profile with admitted uses, not an injective encoding of every Python value. For a new example, `Fraction(3, 4)` and the text `"3/4"` have the same stored representation. No collision attack is involved: the difference is removed before hashing.

Compact sorted JSON determines bytes for the exercised values. Full JCS conformance also requires its primitive serialization and UTF-16 key order. Read RFC 8785, especially Sections 3.2.2 and 3.2.3. Python's sorted keys alone do not meet that contract. The manuscript reports the older encoder; subsequent source changes must be named by their own source pin and receipt.

3. Three distinct comparisons

Let the payload of entry i include its index and predecessor h_(i-1), with the first predecessor equal to the fixed genesis marker. The sink computes `h_i = SHA256(encode(payload_i))`. A header states count n and head h_(n-1).

1. **Local verification** checks the file against its own header and links.
2. **Retained snapshot comparison** compares a verified count/head with a previously held independent pair.
3. **Raw-file comparison** compares a separately held digest of the exact file bytes, including formatting and the header.

Work a fresh three-entry example on paper. Deleting the last entry while leaving count three is locally inconsistent. Deleting it and changing the header to count two can be locally consistent, but differs from the retained three-entry anchor. Adding entry four changes the head too; that inequality does not distinguish legitimate extension from tampering. Verify the whole new chain and compare its entry at retained index two with the old head.

An attacker who controls both the record and the only supposed anchor can replace both. Independent custody is an assumption that must be specified. A self-hash on an anchor does not authenticate its author. Header schema, genesis metadata, whitespace, and normalized type aliases also require care: a matching normalized count/head need not mean matching raw bytes.

4. Read the API's order of effects

`emit_tail_receipt` uses memory. Reopen, or successfully verify the persisted file, before using a receipt as a disk comparison. The check assumes controlled custody and one writer; it does not make a racing writer safe.

The append method updates memory before replacing the file. If replacement fails, memory and disk may diverge. A failure is not an acknowledged append and is not a guaranteed rollback. Reload or discard the old object, establish persisted state, and reconcile the attempted event before deciding on a retry. The package test injects one replacement error; it does not simulate every power failure or filesystem behavior.

Frozen dataclasses protect assignment to their fields, not recursive mutation of contained dictionaries. A later write can expose such mutation by failing verification. API discipline is not immutable media.

For four equally sized 10-byte records, individually rewriting the history writes `(1+2+3+4)*10 = 100` entry bytes; the final snapshot contains 40. This idealized count excludes headers and does not measure actual performance. Bound input bytes, memory, total retained data, writer coordination, and failure recovery separately.

5. Read a candidate catalog as a dataset

Use the snapshot provenance and the package's catalog tests to reproduce file counts offline. The `candidate` and `active` fields are labels in a historical file. A negative-fixture field containing a comment is not an executable failing test. A digest of identifying text is not a bound reproduction receipt.

Keep the evidence ladder explicit: retained event, independently checked reproduction, proposed detector, valid near-miss check, admission for a declared scope/action, and actual host application. Each transition needs its own evidence. The paper proposes this procedure; it reports no integrated semantic miner or measured discovery yield. A predecessor chain supplies an order, not arbitrary dependency edges or semantic equivalence.

The display `a>a` illustrates why scope matters. Under a declared irreflexive relation it is a defect. Under a separately declared loop-permitting grammar it can be valid. The classroom example needs no other paper or repository worksheet to define those two contracts.

6. Interpret maintenance records carefully

Clonal checking has three outcomes: a panel match yields a kill record; no match yields no record; an invalid expression yields a typed refusal. Looking only at the Boolean loses the distinction between a clean check and a refused check. Neither a kill record nor a regulatory record proves that a consumer mutated the repertoire or applied an enforcement policy.

For an explicit list of four pairs with one pattern-only case, toxicity is 1/4, which meets the inclusive 1/5 threshold. Pairs with neither signal still count in that interface's denominator. In benign corpus mode, two matches among eight prompts produce two pattern-only triggers divided by two total triggers, hence

1. That score is a screening policy, not 2/8 or an oracle false-positive estimate. No supplied triggers gives zero by convention and supplies no sample evidence.

7. Exact decay has a separate expiry policy

With fixed positive tau, one idle advance from t to t' applies `k = floor(t'/tau) - floor(t/tau)` halvings. For a score 3/7, idle zero, tau four, and 17 further idle runs, k is four and the new score is 3/112. With expiry at 20 runs, the rule remains active. Expiry and the spacing of score halvings are different parameters when tau is supplied separately.

Calls of 5, 5, and 7 runs cross one, one, and two new intervals. Their sum is four because intermediate floor terms cancel. This proof assumes fixed tau, a nonnegative initial score, and no intervening stimulation or reset. Stimulation with boost 1/5 changes 3/112 to 127/560, resets idle to zero, and returns an active rule. Arithmetic maintenance does not establish admission.

8. Replay and record the actual scope

From the deposit folder, install the documented test tools in a separate environment, or use an existing environment with the same versions:

```
python -m pip install "pytest==8.3.5" "ruff==0.14.13"
python -m pytest code/tests -q -ra
python -m ruff check code
python code/landau_worksheet.py
python verify_deposit.py
python tools/rebuild_release.py --check
```

On Windows, `py -3.12` can replace `python`. The worksheet uses only the standard library and the pinned data snapshot. Its calculations are educational examples, not a reimplementation of the production sink or JCS.

The catalog tests work from the deposit alone. The worked storage tests need the repository source at the expected path and state why they skip when it is absent. The three toy tests are illustrations. Record passed and skipped counts separately and bind the loaded source bytes; do not turn a current checkout pass into a historical-baseline claim.

For effectiveness, group duplicate source cases before splitting and reserve independently checked valid eligible proposals for the safety denominator. Record actual suppression actions, not pattern hits alone. New domains need new positive and valid near-miss evidence plus a stated evaluation plan. Manifest verification certifies the declared byte census. It does not prove detector usefulness, authorize reuse, or create a Zenodo record.

Instructor key: Paper 006

AuthorityEffect: None. The quiz has 24 questions across levels 0-7 and a 96-point rubric. Accept equivalent calculations when their source, encoding, and evidence limits agree. This key is separate from the study guide so learners can attempt the examination first.

Level 0

Q01

The sink assigns index/timestamp, normalizes supplied data, computes digests, and writes a header and entries. It did not calculate the native functional or decide the truth of the supplied reason. `NO_NATIVE_PORT` describes missing infrastructure under its caller's contract, not a demonstrated mathematical defect. Bind checker/source or binary, actual input/output bytes, and an independent verification of the relationship before making a result claim.

Q02

Logical append is the interface's increasing ordered history. The physical file is replaced, not immutable. Successful focused restarts demonstrate the exercised reconstruction checks. No lock, explicit fsync, or power-loss campaign establishes concurrent-writer safety or general durability.

Q03

The value is the literal string `"None"`; it is not JSON `null`. A different entry marker is rejected by creation/reload checks. The required value grants no operational authority and authenticates no producer. A copied marker is not a signature.

Level 1

Q04

`Fraction(2, 6)` reduces to `Fraction(1, 3)` and is stored as `"1/3"`. The text value is already the same string. The protected value cannot distinguish their original types. Retain independently authorized original bytes and an explicit type/encoding contract if exact reconstruction matters.

Q05

The sanitized result is `{"1": "text"}`: the later string key overwrites the earlier key after both become `"1"`. Hashing occurs after this loss. It is a normalization collision, not two different encoded payloads with a SHA-256 collision. Specify allowed key types or retain the original representation.

Q06

U+1F600 has UTF-16 units D83D DE00, whose first unit is less than FB33; therefore it sorts before U+FB33 in JCS. Python code-point order is the reverse. The baseline encoder also permits NaN/infinity and does not implement JCS number serialization. Either is an acceptable second distinction. Later float tagging and rejection use different source bytes; bind them separately.

Level 2

Q07

Change	Local reload	Independent `(2,h1)`	Independent original raw-file digest
Protected reason changed, old digest kept	Integrity rejection	No verified altered snapshot	Differs
Final entry removed, old header kept	Truncation rejection	No verified altered snapshot	Differs
Final entry removed, count/head rewritten	Can pass with `(1,h0)`	Differs	Differs
Only header schema/genesis changed	Can pass at the baseline	Can match	Differs

The anchor must be held outside the rewriting adversary's control, and the argument assumes an alternative protected encoding cannot replace the expected digest. No conclusion about original types, producer honesty, or raw identity follows from normalized equality alone. For the schema case, the baseline loader's omission is part of the answer; the receipt's own schema does not validate the file header.

Q08

No. A legitimate extension also changes its head. Verify all three entries and compare entry index 1's digest with `h1` under the retained count two. Under the hash/custody assumptions this checks the normalized retained prefix. It does not authenticate the producer, the clock, unbound header fields, raw bytes, or the semantic correctness of the additional event.

Q09

The object emits its cached entry count/head, so it has not observed the file change. Reopen and emit only after successful verification, or call `verify_integrity` successfully before emission. The described alteration raises `LedgerIntegrityError`, yielding no verified changed snapshot. Assume one writer and custody that prevents a competing modification during the check.

Level 3

Q10

The exception is not an acknowledged append and not a rollback certificate. The cached receipt counts three; the reopened persisted file still counts two. Discard or successfully reload the old object and reconcile the attempted operation before retrying. After a successful reload the old object also returns to persisted count two. This particular injected failure does not settle all storage failures or crash behaviors.

Q11

No transitive immutability follows. The changed mapping can be written with its stale stored digest during a later append; subsequent reload rejects the changed protected payload. Frozen fields and a tuple do not freeze nested dictionaries. Verified bytes are the evidence source; reference discipline and immutable retention are additional obligations.

Q12

`1000 * (100*101/2) = 5,050,000` accumulated entry bytes, versus `1000*100 = 100,000` final entry bytes. Headers are excluded. Appropriate additional limits include record size, total retained bytes, memory, writer coordination, temporary-path interference, regex runtime, and write-failure recovery. This is an idealized cost count, not empirical throughput.

Level 4

Q13

There are 12 non-placeholder entries. Fractions are 5/61 labelled active and 49/61 carrying placeholders. These counts establish zero demonstrated reproductions or enforcement actions by themselves; this means no such demonstration follows from this evidence, not a measured count of failures of every possible detector. The catalog has 14 literal failure classes.

Q14

The digest covers the specified identifying string, not a reproduction receipt. Raw-event identity would need defined event bytes and their binding; semantic defect identity needs an additional declared equivalence/checker. An independently retained digest permits comparison of its covered bytes. Repeating one record 1,000 times adds stored copies, not 1,000 independently checked defects or draws from a population.

Q15

Under the irreflexive relation, the loop fails the declared predicate. Under the loop-permitting signed grammar, the same display is valid. Bind carrier, grammar, predicate, checker, and route/action scope; retaining the first refusal does not license universal suppression of that phrase. These two classroom declarations suffice without invoking an undeclared graph filling.

Level 5

Q16

True plus a kill receipt reports a benign-panel match and proposes the clone/parent purge. False plus no record reports no match on that panel. False plus an invalid-pattern refusal rejects the malformed clone while sparing its parent under this check. The Boolean alone is inadequate. Returning a record does not mutate either frozen object or prove a repertoire or host-policy change. A consumer must apply and record the authorized action.

Q17

The API index is 1/6, below 1/5, so no downgrade record is synthesized at the default threshold. Conditional on a pattern match, the fraction is 1/3, which would exceed 1/5, but that is not this interface's denominator. Signal presence is not an independent validity label or an observed suppression action. Population false suppression requires independently valid eligible proposals, actual actions, and an appropriate sampling/evaluation plan.

Q18

Corpus mode reports one pattern-only trigger, one total trigger, and toxicity 1. It synthesizes a downgrade at default tau. Empty explicit triggers give counts zero/zero and index zero by convention; this is no-sample evidence. Neither value alone certifies deployment safety, and a returned diagnostic record alone does not change enforcement. Distinguish pattern screening, oracle labels, and applied actions.

Level 6

Q19

Idle advances from 4 to 12. New completed intervals are `floor(12/5)-floor(4/5) = 2-0 = 2`; the score is `(4/5)/4 = 1/5`. Activity is false because `12 < 10` is false. Tau controls score spacing; `half_life_runs` controls idle expiry in this implementation.

Q20

Batch endpoints are 0, 3, 7, 12. Completed interval increments are 0, 1, 1; the successive scores are 1, 1/2, 1/4. One 12-run call crosses two intervals and also yields 1/4. In general, `sum(floor(t_j/tau)-floor(t_(j-1)/tau)) = floor(t_final/tau)-floor(t_initial/tau)`. The intermediate terms cancel. Keep tau fixed, elapsed counts valid, initial score nonnegative, and no resets/stimulation/manual score updates between batches. Changing those conditions changes the claim.

Q21

The score is `1/4 + 1/5 = 9/20`; idle is zero; activity is true. The method returns the modified candidate according to its maintenance policy. It does not independently validate an admission, prove useful reasoning, or show that a host installed the result.

Level 7

Q22

An acceptable packet binds original authorized bytes/encoding, event and host decision, source/checker or binary version, input/output hashes, independently retained anchor and custody, deterministic reproduction and result, a valid near miss with its independent judgment, predicate, fixture bank, permitted action, route/domain, and receipt for the new evaluation. Identifier hashes and placeholder strings leave these bindings/results missing. To demonstrate enforcement, record the consuming host's applied action against the admitted rule version and relevant before/after state, not merely a proposed downgrade.

Q23

Group duplicates/base contents and related source cases before train/holdout allocation. Freeze the detector before held-out evaluation; do not reuse its training near misses as independent tests. Count independently valid eligible proposals in the safety denominator and actual wrongful suppressions in its numerator. Report group diversity and justified dependence handling rather than claiming 1,000 independent samples. A usefulness outcome could be correctly reproduced held-out defects or successful repairs; harmful suppression concerns

valid eligible work stopped by the policy. New domains need their own checked cases/near misses and renewed scope/evaluation evidence.

Q24

Accept: "Three educational checks passed; eleven source-dependent checks were skipped. The PDF compiled and the declared file hashes matched. These results do not execute the production behavior, establish detector effectiveness, or create a Zenodo publication." State the skip cause, needed source checkout and test executions, exact hashes, and publication status. Hash identity, exercised behavior, effectiveness, and external publication are separate claims. An honest methods preprint can state those limits.

Replay and rubric

Run the standard-library exact worksheet:

```
python code/landau_worksheet.py
```

It recalculates the quiz's fractions, batching endpoints, cost count, normalization example, key-order counterexample, and catalog counts. It does not replay the production sink, certify JCS, or run a detector. The repository-dependent storage tests supply the separately scoped behavior checks. Follow the study guide for both kinds of replay.

Award one point each for result, derivation/counterexample, declared contract, and evidence limit. Require at least 3/4 on every question in each completed level, with the quiz's evidence-boundary corrections before advancement. Alternative valid packets or experiments receive credit when they cover the same obligations without inventing observed results.

Theoretical minimum: Paper 006

AuthorityEffect: None. Read the fictional preface and study guide. Keep the instructor key closed until completing the questions. Give exact fractions, explicit source or encoding assumptions, and the strongest conclusion the evidence permits. These exercises concern the manuscript's audited commit, not every later branch.

Level 0: name the object before making a claim

Q01. Event, reason, and truth

A stored entry says `NO_NATIVE_PORT` and contains `{"phi1": "0"}`. Which parts of this record were computed by the sink? Can the record establish that the proposed mathematical witness is invalid, or that a native functional was zero? Identify the missing bindings.

Q02. Four persistence claims

An API exposes append, writes a temporary copy of all records, and calls `os.replace`. Distinguish logical append, physical immutability, successful process restart, and power-loss durability. Which are supported by the source and the focused restart tests?

Q03. A fixed marker

An entry and a tail receipt contain `authority_effect: "None"`. Is that value JSON `null`, a cryptographic signature, or an authorization? What happens if the required entry marker is replaced with another value? Does keeping the correct marker authenticate the event's producer?

Level 1: state exactly which bytes are protected

Q04. A lossy value conversion

The baseline sanitizer maps a fraction to its reduced rational string. Compare `Fraction(2, 6)` and the literal string `"1/3"`. Can the stored value distinguish them? What must be retained to reconstruct the original type?

Q05. Colliding mapping keys

Normalize the insertion-ordered mapping `{1: "numeric", "1": "text"}` using the baseline's key stringification. What remains? Explain why no SHA-256 collision is needed for this loss of information.

Q06. Sorted JSON and JCS

Python code-point ordering places U+FB33 before U+1F600. JCS uses UTF-16 code units. Which key comes first under JCS? Give a second difference between the baseline encoder and JCS. Explain why the current branch's float-tag change cannot silently be counted as a historical-baseline result.

Level 2: compare verified snapshots

Q07. Four modifications

Retain an independent anchor `(2, h1)`, where `h1` is the digest of entry index 1. Classify each modification under (a) local reload, (b) the retained count/head comparison, and (c) an independently retained raw-file digest:

- Change entry 0's protected reason but keep its old digest.
- Remove entry 1 while keeping the original header.
- Remove entry 1 and rewrite count/head to `(1, h0)`.
- Change only the header schema and genesis timestamp.

State the hash and custody assumptions behind the classification.

Q08. A legitimate extension

A fully verified three-entry file has head `h2`, while the retained anchor is `(2, h1)`. Does inequality of heads prove tampering? What additional prefix comparison can establish extension of the retained normalized history? What does even that comparison leave unproved?

Q09. Cached evidence

Open a verified two-entry sink. An external writer changes its on-disk reason without rehashing. The old object emits the same tail receipt. Why? Give a correct disk-check sequence and explain its custody/concurrency assumption.

Level 3: failed writes and resource obligations

Q10. An unacknowledged append

An append advances memory from two to three entries; replacement then raises `OSError`. Disk still has two entries. Classify the exception, the old object's three-entry receipt, and the reopened file. What must happen before retrying?

Q11. Frozen outside, mutable inside

The sink returns a frozen dataclass containing a dictionary. A caller edits that dictionary; a later append writes the changed data with the old digest. What can a subsequent reload do? Does a tuple of frozen entries provide transitive immutability?

Q12. Cost of individually rewriting the history

Ignore headers and suppose each record contributes exactly 1,000 bytes. Compute accumulated entry bytes written over 100 individual appends and the final snapshot's entry bytes. Identify two additional operational limits that the checksum chain does not impose. Do not call this calculation a benchmark.

Level 4: a catalog is data, not an admitted repertoire

Q13. Census without promotion

The pinned catalog has 61 entries: 56 labelled `candidate`, 5 labelled `active`, and 49 with placeholder fixture pairs. Compute the non-placeholder count and the exact fractions labelled active and carrying placeholders. How many reproduced or enforced detectors do these counts establish?

Q14. An evidence-named hash

An `evidence_sha256` is SHA-256 of `defect_label:run_id:task_id`. Does it bind a reproduction receipt? Distinguish identity of these encoded identifiers, identity of a raw event, and semantic identity of a defect. Explain what 1,000 copies of one record add to independent evidence.

Q15. One display, two contracts

In one declared relation, self-loops are forbidden; in a separate signed-move grammar, loops are legal. The display `a>a` is retained from both routes. Give the reproduced defect and valid near miss, and explain what prevents a phrase detector from transferring the first route's judgment to the second.

Level 5: decisions, denominators, and actual host action

Q16. Three clonal outcomes

Classify `(True, KillConditionReceipt)`, `(False, None)`, and `(False, KillConditionRefusal(code="INVALID_MATCH_PATTERN"))`. Which result rejects the malformed clone without purging its parent? Does returning a kill record mutate either frozen object or apply a host policy?

Q17. The denominator changes the answer

The explicit trigger interface receives one `(True, False)`, two `(True, True)`, and three `(False, False)` records. Compute its toxicity and compare with inclusive tau = 1/5. Also compute the fraction conditional on a pattern match. Why is neither value automatically the population false-suppression rate on independently valid eligible proposals?

Q18. Corpus mode and no evidence

Exactly one of 100 benign prompts matches in corpus mode. What are `signal1_only_count`, `total_triggers`, and toxicity? Contrast this with an empty explicit trigger list. Can either number certify deployment safety? Does a synthesized `DIAGNOSTIC_ONLY` record by itself change enforcement?

Level 6: exact maintenance arithmetic

Q19. Decay from an already idle rule

A rule has score 4/5, idle count 4, and expiry `half_life_runs=10`. Apply eight further idle runs with tau = 5. Compute completed new intervals, new score, new idle count, and `is_active`. Distinguish decay spacing from expiry.

Q20. Prove batching invariance with its assumptions

Starting at idle zero with score 1 and fixed tau = 5, compare one call for 12 runs with calls for 3, 4, and 5 runs. Compute every interval increment and the final score, then prove the telescoping formula for arbitrary batches. State conditions under which this proof no longer applies.

Q21. Stimulation resets the schedule

An inactive rule has score 1/4 and idle count 12. Apply stimulation with boost 1/5. Compute score, idle count, and activity according to the baseline method. Does this result validate a new detector's admission or empirical usefulness?

Level 7: make a publication claim reviewable

Q22. A minimum reusable-evidence packet

Design a packet for promoting one retained event into a candidate detector. Specify event bytes, source/checker bindings, independent anchor custody, reproduction, a valid near miss, predicate, action, and scope. Mark which items are missing from an identifier-hash-only catalog entry. What receipt would be needed to show a host actually applied an admitted action?

Q23. A held-out experiment

A benign panel contains 1,000 rows generated from 40 base contents. Design a split and denominator that do not present repetitions or training examples as independent safety evidence. State one usefulness outcome and one harmful suppression outcome to measure. What would justify extending a claim to a new domain?

Q24. Release audit

A report says three educational tests passed and eleven production-dependent tests skipped. A PDF compiled and a manifest matched. Write an honest release statement, list the missing execution evidence, and separate byte integrity, source-bound behavior, detector effectiveness, and external Zenodo publication.

Grading and advancement

Each question earns 0-4 points: correct result (1), explicit calculation or counterexample (1), appropriate contract/source assumptions (1), and correct evidence limit (1). Total: 96 points. A level passes when each of its three questions earns at least 3 points. Advancement requires all lower levels to pass. Any claim that a hash authenticates an event, a cached receipt proves disk state, or a decision record proves enforcement must be corrected before advancement. The instructor key supplies acceptable answers and corrections.

Scores assess these exercises only; they confer no operational authority, historical Landau credential, or measured learning gain.